深度学习项目

深度学习原理及应用

郭睿

第11周

深度学习项目 vs 传统软件工程

定义

以数据驱动的工程活动，贯穿采集、处理、模型设计、训练与部署。侧重于实验、迭代与性能优化。

与传统软件相比

传统软件由规则驱动，结果可预测；深度学习由数据驱动，输出概率化且难以解释。工程目标不再是无 bug，而是性能优化与监控。

深度学习工程 vs 传统软件工程

对比维度	深度学习工程	传统软件工程
驱动力	数据与经验	需求与规则
可控制性	结果概率化，黑盒特性	结果确定，逻辑清晰
主要风险	数据偏差、过拟合	代码缺陷、系统故障
调优策略	改进数据与模型	优化算法与代码

什么是深度学习项目？

- **项目源头是数据（Data）而不是代码（Code）**
- **结果取决于模型 + 数据 + 训练流程 + 部署环境**
- 需要不断观察系统表现并持续改进（Iterative & Empirical）

什么是深度学习项目？

- 案例 1：人脸识别开门系统
  - 任务：判断门口的人是否是授权用户（“能不能给他开门？”）
    涉及步骤：
  - 采集不同光照、不同姿态的人脸
  - 训练 Siamese 模型判断两张脸是否同一人
  - 部署到门禁系统
  - 监控冬天人们戴围巾后识别率变化
  - → **典型的端到端深度学习系统，从数据到部署都涉及。**

什么是深度学习项目？

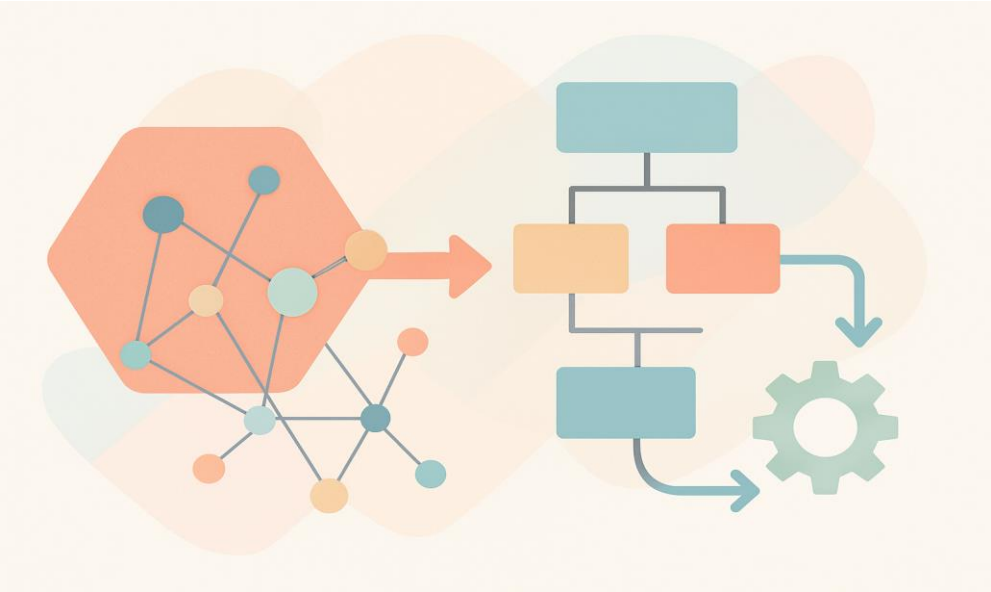
- **案例 2：语音唤醒词系统（Wake Word）**
  - 任务：检测“Robert, turn on”这样的唤醒词
    流程包括：
  - 收集多人语音
  - 处理不平衡样本（正样本很少）
  - 合成背景噪声语音
  - 将模型部署到低功耗芯片上
  - 持续更新数据以适应不同环境
  - → **结合模型、数据、硬件的综合工程项目。**

什么是深度学习项目？

- 任务：输入一个问题 → 自动搜索 → 读取文章 → 输出总结

- 典型 pipeline：
  - Query → 生成搜索词 → 进行 Web 搜索 → 抓取网页 → LLM 写总结
- **一个由多个 AI 组件组成的管线（Pipeline）项目。**

完整流程与生命周期



1. **定义任务**
   明确目标与评价指标
2. **收集数据**
   采集并清洗数据
3. **特征/模型**
   设计架构与特征
4. **训练评估**
   优化并验证性能
5. **错误分析**
   找出主要错误类别
6. **快速迭代**
   修改数据与模型
7. **部署上线**
   集成至产品环境
8. **持续监控**
   检测漂移并更新

深度学习项目的完整流程（人脸识别开门举例）

1. 定义任务　Define the Problem　是否应该给来访者开门?
2. 收集数据　Collect Data　两天内收集一个小型数据集即可启动
   公司前台摆一个相机，让愿意的同事采集几百张脸。
3. 设计模型　Design the Model　Siamese / Embedding 模型
   （两张脸 → 计算是否是同一人）
4. 训练模型　Train the Model　超参数调整（lr、batch size 等）
5. 误差分析　Error Analysis　模型在弱光下表现差 → 收更多弱光照片
   模型对戴口罩的人误判多 → 收更多戴口罩的照片
6. 快速迭代　Rapid Iteration Loop　早上：查看昨天训练结果　中午：做误差分析　下午：修模型或补数据
   晚上：重新训练
7. 部署上线　Deployment　在设备上运行"轻量级检测"
8. 监控维护　Monitoring & Maintenance　冬天大家戴围巾　用户群体变化

深度学习项目不是"训练一个模型"，而是：
从数据 → 模型 → 迭代 → 上线 → 持续维护的完整生命周期。

深度学习项目强调“速度”

为什么速度重要?

- 错误难以预见，只有真正训练后才能暴露问题。
- 快速迭代=更多实验，增加成功机会。
- 数据才是关键：通过改善数据质量来提升模型。
- 切记：不要沉迷调参，先解决数据问题！

人脸识别开门系统

- Day 1 晚上：训练模型
- Day 2 早上：查看结果 → 误差分析
- Day 2 下午：修复一个问题
- Day 2 晚上：重新训练

语音唤醒词系统

- 定义任务（Define the Task）
  - 目标：检测一句话中某个瞬间是否出现了唤醒词。
  - Wake Word： Robert, turn on
- 收集数据（Collect Data）
  - 找同学/同事录音（真实数据最有效）
  - 收一些“非唤醒词”的自然对话作为负样本
  - 合成背景噪声（咖啡厅、街道、键盘声等）

语音唤醒词系统

- 构建训练样本（切片技巧）10秒音频

  [---噪声---][--说语音--][---噪声---]

  ↑唤醒词

  0.0s - 1.0s → 负样本

  1.0s - 2.0s → 负样本

  ...

  5.0s - 6.0s → 正样本（刚说完唤醒词）

第一次训练 & 发现问题

- 训练集：97% 准确率
- 验证集：几乎从不触发唤醒

- 因为数据不平衡：
  模型学会了“一直输出 0”，也能得到很高的准确率。

修复策略（Error → Fix）

- **方法 1：扩充正样本（最有效）**
  - 多录一些 “Robert, turn on”
  - 复制正样本（oversample）
  - 增加窗口数量

- **方法 2：合成数据**
  - 将“唤醒词语音” + “背景噪声” 做混合
  - 创建成千上万条新样本

- **方法 3：修改标签策略**
  - 说完后 0.5 秒内都视为正样本

误差分析（Error Analysis）

1：收集“模型失败的样例”
找出模型“分类错误”的样本
建立一个错误样例清单（10 ~ 100 个即可）

误差分析（Error Analysis）

2：为错误样例分类（标签错误类型）

示例 A：人脸识别开门系统

- 错误可以按以下分类：
- 光线太暗（Low light）
- 戴口罩（Mask）
- 背光（Backlight）
- 大侧脸（Side pose）
- 被挡住部分脸（Occlusion）

示例 B：唤醒词检测（Wake Word）

- 错误类型可能是：
- 背景噪声太大（Noise）
- 人讲话太轻（Low volume）
- 句子速度太快（Fast speech）
- 说的不是唤醒词却触发（False Trigger）
- 说了唤醒词却没触发（Missed Trigger）

误差分析（Error Analysis）

3：统计各类错误的比例（哪类错误最多？）

错误类型	错误数量	占比
背景噪声（Noise）	18	45%
语速太快（Fast speech）	6	15%
音量太小（Low volume）	2	5%
假触发（False trigger）	8	20%
其他	6	15%

- 背景噪声太大 → 造成错判
- 用户说话太快 → 造成错判
- 用户声音太小 → 造成错判
- 干扰语音太多
- 口音不同

误差分析（Error Analysis）

4：找"最值得修"的前 1-2 个问题

45% 的错误来自背景噪声 → **优先补背景噪声数据**

方案 A：收集真实背景噪声数据

- 咖啡厅的噪声
- 宿舍环境噪声
- 教室里写字声、键盘声
- 街道噪音
- 室内空调持续声
- 家庭环境中的电视声

方案 B：合成背景噪声数据

- 纯净的唤醒词录音（clean wake word）
- 大量背景噪声（noise samples）

new_audio = wake_word_audio + background_noise

反复迭代（Iteration）

- 第一天训练 → 不触发
- 第二天：修正标签
- 第三天：扩充正样本
- 第四天：加入背景噪声
- 第五天：模型开始成功触发

工程化方法论总结

- 深度学习项目 = 数据 + 模型 + 迭代 + 部署

  深度学习项目不是训练一个模型，而是一个
  从数据 → 模型 → 错误分析 → 迭代 → 上线 → 维护
  的端到端工程流程。

- **1. 项目始于数据（Data First）**
  - 数据比模型更重要
  - 先快速构建一个"可运行的原型"
  - 根据错误分析补数据，而不是盲目扩大规模

工程化方法论总结

- 深度学习项目 = 数据 + 模型 + 迭代 + 部署

  深度学习项目不是训练一个模型，而是一个
  从数据 → 模型 → 错误分析 → 迭代 → 上线 → 维护
  的端到端工程流程。

- **2. 迭代速度决定成败（Iteration Speed = Key）**
  - 每日循环（train → analyze → fix → train）
  - 小步快跑比一次造大数据集更有效
  - 工程团队速度差 2 倍，效果差可能 10 倍

工程化方法论总结

- 深度学习项目 = 数据 + 模型 + 迭代 + 部署

  深度学习项目不是训练一个模型，而是一个
  从数据 → 模型 → 错误分析 → 迭代 → 上线 → 维护
  的端到端工程流程。

- **3. 错误分析是最重要的技能（Error Analysis）**
  - 找出“最值得修”的 1–2 个错误类型
  - 聚焦问题，而不是到处改
  - 精准补数据 → 最大化收益

工程化方法论总结

- 深度学习项目 = 数据 + 模型 + 迭代 + 部署

  深度学习项目不是训练一个模型，而是一个
  从数据 → 模型 → 错误分析 → 迭代 → 上线 → 维护
  的端到端工程流程。

- **4. 工程现实：模型上线后还会退化**
  - Data Drift / Concept Drift
  - 季节变化、用户行为变化、设备变化
  - 必须监控 & 持续更新

工程化方法论总结

- 深度学习项目 = 数据 + 模型 + 迭代 + 部署

  深度学习项目不是训练一个模型，而是一个**从数据 → 模型 → 错误分析 → 迭代 → 上线 → 维护**的端到端工程流程。

- **5. 深度学习项目是一种"实验型工程"**
  - 不同于传统软件：
  - 行为不是由代码定义，而是由数据定义
  - 不能预测模型会犯什么错
  - 但可以通过迭代逐步修复所有错误